

Coordinated Multi-Technician Jobs

This feature is fundamentally about making **multiple technicians arrive at the same location at approximately the same time**, while still allowing each technician to have their own vehicle, skills, service duration, availability, and load.

Three things to prove:

1. The API accepts and validates the new structure correctly.
2. The optimizer actually creates a coordinated crew correctly.
3. The optimizer never partially assigns or incorrectly synchronizes a crew.

1. First — understand the feature in simple terms

Normally:

```
None
Job 101
  ↓
Vehicle 1
  ↓
Customer
```

One job = one vehicle.

But imagine an installation that requires:

```
None
Electrician + HVAC Technician
      ↓
    SAME CUSTOMER
```

You need:

```
None
Vehicle A —————→ Customer
                        ↑
Vehicle B —————→ Customer
```

And not:

None

Vehicle A → Customer at 10:00

Vehicle B → Customer at 11:00 ❌

The new `coordinate_jobs` feature guarantees that the technicians assigned to one group:

- use **different vehicles**
- arrive within the configured tolerance
- satisfy their individual skills/constraints
- are assigned **all together or not at all**

The document explicitly calls this out as the core guarantee.

2. What does `coordinate_jobs` look like?

It is a **new top-level field**.

JSON

```
{
  "jobs": [...],

  "coordinate_jobs": [
    {
      "id": 501,
      "location_index": 12,

      "jobs": [
        {
          "id": 5011
        },
        {
          "id": 5012
        }
      ]
    }
  ]
}
```

Think of it as:

```
None
coordinate_jobs
├── Group 501
│   ├── Technician/job 5011
│   └── Technician/job 5012
```

The group itself is **not a route step**.

Each member becomes an ordinary **job** step in a vehicle route. The common identifier is **coordination_group_id**.

3. The most important concept: GROUP vs MEMBER

This is where I think you should spend most of your understanding.

Group

The group defines what is common:

```
JSON
{
  "id": 501,
  "location_index": 12,
  "skills": [7],
  "service": 3600,
  "time_windows": [[28800, 61200]],
  "arrival_sync_tolerance_seconds": 300
}
```

Meaning:

I need a crew at location 12, everyone needs skill 7, default service is 1 hour, they must arrive within 5 minutes.

Members

Then:

JSON

```
"jobs": [  
  {  
    "id": 5011,  
    "skills": [20]  
  },  
  {  
    "id": 5012,  
    "skills": [21],  
    "service": 1800  
  }  
]
```

Now:

Member 5011

Needs:

None

Group skills: 7

+

Own skills: 20

=

Vehicle must have {7,20}

Member 5012

Needs:

None

Group skills: 7

+

Own skills: 21

=

Vehicle must have {7,21}

This **skill union** behaviour is explicitly defined in the design.

4. Technician count

This is simple:

None

```
jobs.length = number of technicians required
```

So:

JSON

```
"jobs": [  
  {"id": 5011},  
  {"id": 5012}  
]
```

means:

I need 2 technicians.

Three members:

None

```
5011  
5012  
5013
```

means:

I need 3 technicians.

Minimum is 2.

5. Very important: different vehicles

If there are 2 technicians:

None

```
5011 → Vehicle 1  
5012 → Vehicle 2
```

Correct.

This is NOT allowed:

None

5011 → Vehicle 1

5012 → Vehicle 1

The distinct-vehicle requirement applies **within the group**.

But this is allowed:

None

Morning

Group 501:

5011 → Vehicle 1

5012 → Vehicle 2

Afternoon

Group 777:

7771 → Vehicle 1

7772 → Vehicle 3

The same vehicle can participate in **different groups at different times**.

6. The most important part — arrival synchronization

This is the heart of the feature.

Suppose:

None

Tolerance = 300 seconds

That's:

None

5 minutes

And:

None

Technician A arrives = 10:00

Technician B arrives = 10:03

Difference:

None

3 minutes



PASS

But:

None

Technician A = 10:00

Technician B = 10:08

Difference:

None

8 minutes



FAIL

The maximum spread is:

None

latest arrival - earliest arrival

and it must be $\leq$ tolerance.

7. Why this is more complicated than a normal time window

This is an important QA concept.

Suppose:

None

Tech A can arrive: 09:00–11:00

Tech B can arrive: 10:00–12:00

Their common possible period is:

None

10:00–11:00

So they can meet.

But suppose:

None

Tech A: 09:00–09:30

Tech B: 10:00–10:30

They cannot meet.

Even though both individually have valid time windows.

That's why this feature needs actual **synchronization**, not just precedence. The design explicitly discusses this two-sided feasibility concept.

8. arrival_sync_tolerance_seconds

Default:

None

300 seconds

= 5 minutes.

You can override it:

JSON

```
"arrival_sync_tolerance_seconds": 600
```

= 10 minutes.

Or configure request-level default:

JSON

```
"options": {  
  "coordination": {  
    "default_arrival_sync_tolerance_seconds": 600  
  }  
}
```

Group-level value should override the request-level default.

The documented defaults are 300 seconds.

9. service_start_rule

There are two options.

last_arrival — default

Suppose:

None

Tech A arrives: 10:00

Tech B arrives: 10:03

Tolerance = 5 minutes.

Under:

JSON

```
"service_start_rule": "last_arrival"
```

Both start at:

None

10:03

Tech A waits:

None

3 minutes

The design says that waiting should appear in `waiting_time`.

`first_arrival`

Now:

None

A arrives 10:00

B arrives 10:03

They can each start when they arrive.

So:

None

A service → 10:00

B service → 10:03

This is for cases where they only need to be present together rather than start work simultaneously.

10. Different service durations

This is another important scenario.

Example:

None

Electrician = 3600 sec

Inspector = 1800 sec

Under `last_arrival`:

None

A arrives 10:00

B arrives 10:03

```
Both start 10:03
```

Then:

```
None
```

```
Electrician:  
10:03 → 11:03
```

```
Inspector:  
10:03 → 10:33
```

So their **departures don't need to be synchronized**.

Only arrival/start coordination is required.

The acceptance walkthrough explicitly describes this scenario.

11. Skills are VERY important to test

Take:

```
JSON
```

```
"group skills": [7]
```

Member 1:

```
JSON
```

```
"skills": [20]
```

Member 2:

```
JSON
```

```
"skills": [21]
```

Effective requirements:

None

Member 1 $\rightarrow 7 + 20$

Member 2 $\rightarrow 7 + 21$

You need:

None

Vehicle A $\rightarrow 7 + 20$

Vehicle B $\rightarrow 7 + 21$

Having two vehicles with only:

None

Vehicle A $\rightarrow 20$

Vehicle B $\rightarrow 21$

should NOT be enough.

12. Pinned technician

This is very useful.

You can pin a member:

JSON

```
"relations": [
  {
    "type": "pinned",
    "vehicle": "crane-op-1",
    "steps": [
      {
        "type": "job",
        "id": 7771
      }
    ]
  }
]
```

Meaning:

None

Member 7771

↓

MUST use crane-op-1

Other members can be optimized normally.

The design explicitly uses the existing pinned relation mechanism for this.

13. Response — what should you look for?

Suppose:

None

Group = 501

Members:

5011

5012

You should get:

JSON

```
{
  "type": "job",
  "id": 5011,
  "coordination_group_id": 501
}
```

and:

JSON

```
{
  "type": "job",
  "id": 5012,
  "coordination_group_id": 501
}
```

So:

None

`id`

↓

`identifies technician/job`

`coordination_group_id`

↓

`identifies the crew`

The group itself does not appear as a step.

14. Your core response assertions

For a successful 2-person group, I would assert:

Assertion 1

Both members exist.

None

`5011 exists`

`5012 exists`

Assertion 2

Both have:

None

`coordination_group_id = 501`

Assertion 3

They are on different vehicles.

None

`vehicle(5011) != vehicle(5012)`

Assertion 4

Same location.

None

```
location_index(5011) == 12  
location_index(5012) == 12
```

Assertion 5

Arrival difference:

None

```
abs(arrival1 - arrival2) <= tolerance
```

Assertion 6

If `last_arrival`:

None

```
service_start1 == service_start2
```

Assertion 7

Waiting time is correct for the early technician.

15. The most important negative test

This is probably your **#1 failure scenario**.

Create:

None

```
2 technicians
```

```
2 vehicles
```

but make their routes such that:

None

```
Vehicle 1 can reach site at 10:00  
Vehicle 2 can reach site at 10:20
```

Tolerance:

None

300 sec = 5 min

Difference:

None

20 min

Expected:

None

WHOLE GROUP = UNASSIGNED

Not:

None

Tech 1 = assigned

Tech 2 = unassigned

And not:

None

Tech 1 = assigned

Tech 2 = assigned

with a 20-minute gap.

The acceptance section explicitly says that a 20-minute feasible gap with a 5-minute tolerance results in unassigned, while increasing tolerance to 25 minutes makes it schedulable.

16. Another critical negative test: insufficient technicians

Group:

None

3 technicians

But only:

None

2 eligible vehicles

Expected:

None

Group = unassigned

Response should explain something like:

JSON

```
"required_technicians": 3,  
"best_achieved": 2
```

And each member appears in `unassigned[]`.

This is explicitly defined in the design.

17. Test matrix I recommend

Basic

TC	Scenario	Expected
01	2 members, 2 vehicles	Assigned
02	3 members, 3 vehicles	Assigned
03	4 members, 4 vehicles	Assigned
04	Minimum 2 members	Works
05	1 member	400
06	Missing group ID	400
07	Missing location	400

Vehicle assignment

TC	Scenario
08	Enough eligible vehicles
09	Insufficient vehicles
10	Exactly enough vehicles
11	Same vehicle eligible for all members
12	Verify same vehicle is never used twice within group
13	Pinned member
14	Two pinned members to different vehicles
15	Two members pinned to same vehicle → 400

Synchronization

TC	Scenario
16	Arrivals exactly simultaneous
17	Arrivals 1 sec apart
18	Arrivals exactly at tolerance
19	Arrivals 1 sec beyond tolerance
20	Arrivals 20 min apart, tolerance 5 min
21	Tolerance increased to 25 min
22	Tolerance = 0
23	Different member time windows
24	Overlapping windows
25	Non-overlapping windows

18. Service start testing

Last Arrival

Test:

None

A → 10:00

B → 10:03

Expected:

None

A waiting = 180 sec

A service starts = 10:03

B service starts = 10:03

first_arrival

Expected:

None

A service starts = 10:00

B service starts = 10:03

The design defines both behaviours.

19. Skills test matrix

I would specifically test:

None

Group skills = [7]

Member A = [20]

Member B = [21]

Then:

Vehicle	Skills	Expected
V1	7,20	A eligible
V2	7,21	B eligible
V3	7	Neither
V4	7,20,21	A/B eligible

Then test whether the solver chooses the correct combination of distinct vehicles.

20. Time-window test that is particularly valuable

Member A:

None

08:00–17:00

Member B:

None

10:00–12:00

Expected:

The entire crew should be scheduled within the feasible common period driven by B's availability.

The design specifically gives this as an example: the inspector's narrower 10:00–12:00 window can drive the meeting time for the whole crew.

21. Loads

Unlike a normal job, **group-level load does not exist**.

Loads belong to individual members:

JSON

```
{
  "id": 5011,
  "delivery": [10]
}
```

or:

JSON

```
{
  "id": 5012,
  "pickup": [20]
}
```

```
}
```

The design explicitly says loads exist only on members because a group-level load would be ambiguous across multiple vehicles.

So test:

- member pickup
- member delivery
- multiple members with different loads
- vehicle capacity
- no group-level load
- invalid group-level load if someone tries to send one

22. Things that are NOT supported

These are easy places to accidentally waste time testing.

✗ Shipment members

Members are job-type visits.

Shipment-style pickup + delivery pairs are **out of scope in v1**.

✗ Nested coordination groups

```
None
Group A
  ↓
Group B
```

Not supported.

✗ Simulation mode

Excluded from v1.

✗ Live hold/re-plan

If a technician is late in real life, the engine does not hold the others waiting in real time.

You re-run re-optimization with updated inputs.

✗ Fast API

`coordinate_jobs` is Flexible API only.

Fast should return:

None
`400`

23. ID validation

This is another important API test.

Group IDs and member IDs must be globally unique across:

None
`jobs`
`shipments`
`shipment steps`
`groups`
`member jobs`

Example:

None
`Normal job ID = 100`

`Coordinate group ID = 100`

✗ Invalid.

Likewise:

None
`Member ID = 100`
`Normal job ID = 100`

✗ Invalid.

The design explicitly requires global uniqueness.

24. Validation test cases

You should have these as automated 400 tests:

TC	Invalid Input	Expected
V01	Fast API + coordinate_jobs	400
V02	Missing group <code>id</code>	400
V03	Missing <code>location_index</code>	400
V04	Less than 2 members	400
V05	Duplicate group ID	400
V06	Duplicate member ID	400
V07	Member ID duplicates normal job ID	400
V08	Member ID duplicates shipment step ID	400
V09	Negative tolerance	400
V10	Invalid <code>service_start_rule</code>	400
V11	Member referenced in non-pinned relation	400
V12	Group referenced in relation	400
V13	Same vehicle pinned to two members	400
V14	Same member pinned twice	400
V15	Non-distinct pinned vehicles	400
V16	Impossible single-window synchronization	400

The validation table in the design explicitly covers these categories.

25. Multiple groups — VERY important E2E test

Create:

None

Group 501

Technician A

Technician B

Group 777

Technician C

Technician D
Technician E

You need to verify that:

None
Group 501
→ synchronized internally
Group 777
→ synchronized internally

but there is **no requirement that Group 501 and Group 777 happen simultaneously**.

A vehicle can even participate in both groups at different times.

This is explicitly part of the acceptance expectations.

26. Re-optimization

This needs its own test suite.

Initial

None
Group 501
5011 → V1
5012 → V2
Meeting = 10:00

Re-optimize without changing the crew.

Expected:

None
Still coordinated

The previous vehicles seed the initial plan and previous meeting time is honoured unless changes require a new plan.

Remove group

Remove:

None

coordination_group_id = 501

Expected:

None

ALL members removed

Partial removal

Try removing:

None

5011

while leaving:

None

5012

Expected:

None

400

The group cannot be partially removed.

27. Cost testing

An important clarification from the document:

There is **no special discount** for having multiple technicians at the same job.

Each technician's route is costed normally:

None

Technician A

```
→ own drive cost
→ own fixed/hourly cost
Technician B
→ own drive cost
→ own fixed/hourly cost
```

This is explicitly the v1 cost model.

So don't expect:

```
None
2 technicians = one shared route cost
```

28. Dynamic assertions I'd implement

Since you're using API automation, don't assert exact vehicle IDs unless the input pins them.

Instead, calculate dynamically.

Find coordinated members

```
JavaScript
const groupId = 501;

const coordinatedSteps =
  response.data.result.routes
    .flatMap(route =>
      route.steps
        .filter(step =>
          step.coordination_group_id === groupId
        )
        .map(step => ({
          ...step,
          vehicle: route.vehicle
        })))
    );
```

Then:

JavaScript

```
expect(coordinatedSteps.length).to.equal(2);
```

Verify distinct vehicles

JavaScript

```
const vehicles =
  coordinatedSteps.map(step => step.vehicle);

expect(new Set(vehicles).size)
  .to.equal(coordinatedSteps.length);
```

Verify IDs

JavaScript

```
const ids =
  coordinatedSteps.map(step => step.id);

expect(ids).to.include(5011);
expect(ids).to.include(5012);
```

Verify synchronization

Depending on the exact timestamp fields returned by your API, calculate:

JavaScript

```
const arrivalTimes =
  coordinatedSteps.map(step =>
    new Date(step.arrival).getTime()
  );

const spread =
  Math.max(...arrivalTimes) -
  Math.min(...arrivalTimes);

expect(spread).to.be.at.most(300000);
```

Important: confirm the actual response field used for arrival time before implementing this assertion. The design tells us the synchronization requirement, but does not define a new `arrival` response field.

29. Questions to ask Engineering

1. What exact response field represents the member's arrival timestamp?

We need this to automate:

```
abs(arrivalA - arrivalB) <= tolerance
```

2. What exactly does `waiting_time` include?

The design says it includes waiting for the last technician under `last_arrival`, but confirm whether it includes:

- normal route waiting
- time-window waiting
- coordination waiting

or whether those are separated.

3. What exactly happens with member `pickup` / `delivery` loads?

```
Member A delivery = 10
```

```
Member B delivery = 20
```

Do both vehicles independently carry their own load constraints exactly like ordinary jobs?

4. What happens when the solver finds several valid meeting windows?

Which one does it prefer?

- earliest?
- cheapest?
- shortest route?
- least waiting?
- some other optimization objective?

The document says the optimizer explores different meeting times, but doesn't give us the selection objective.

5. Does group-level `service` apply after the synchronized arrival or before it?

The document implies service begins at the last arrival under `last_arrival`, but I'd verify the exact timeline.

6. How is `setup` treated with `last_arrival`?

Does setup happen:

`Before or after synchronization?`

7. Does group-level `time_windows` get intersected with every member's individual window?

The design says member overrides replace the group default, so I would explicitly confirm the exact inheritance logic.

8. If a member has multiple time windows, how does synchronization choose between them?

The document says multi-window cases are decided at solve time rather than by the static validation.

9. Can two members have exactly the same vehicle eligibility?

If yes, how does the optimizer select two distinct vehicles?

10. What happens if only one vehicle satisfies both members' skills?

11. What happens with `arrival_sync_tolerance_seconds = 0`?

The document says it is accepted but rarely satisfiable.

12. Is there a maximum allowed tolerance?

13. Is `service` allowed to be 0?

14. Is `setup` allowed to be 0?

15. Are negative service/setup values rejected?

16. Are negative/empty skills rejected?

17. Is `jobs` allowed to contain duplicate member descriptions?

18. If one member's vehicle becomes unavailable, does the entire group get re-synchronized?

19. If the previous meeting time becomes impossible, does the solver freely choose a new meeting time?

20. Can `shipment_changes` or other existing reoptimization operations reference a coordinate member ID?

Answers to Questions

Q1. What exact response field represents the member's arrival timestamp?

The field is `routes[].steps[].arrival` (seconds, same time base as your input).
However, `abs(arrivalA - arrivalB) <= tolerance` is NOT an invariant the solver guarantees. That assertion will produce false failures.

What the engine bounds is the SERVICE START, i.e. `arrival + waiting_time`.

Real output from a passing case with tolerance = 300:

`veh=10 id=5011 arrival=29400 waiting=20600 => serviceStart=50000`

`veh=20 id=5012 arrival=33800 waiting=16200 => serviceStart=50000`

`raw arrival spread = 4400s >> tolerance 300s`

Both technicians start work at 50000 (the actual requirement for a two-person job), but a shared time window pushed both there, leaving raw arrivals 4400s apart.

Please assert this instead:

`serviceStart(X) = X.arrival + X.waiting_time`

`abs(serviceStart_A - serviceStart_B) <= tolerance`

Under `last_arrival` (the default) this difference is always exactly 0, so you can assert equality directly.

Q2. What exactly does `waiting_time` include?

All of them, summed into one field. It is NOT broken down.

Note that in this engine "normal route waiting" and "time-window waiting" are the same thing: waiting only ever arises from arriving before a time window opens. There is no third category. So:

`waiting_time = time-window waiting + coordination waiting`

Observed: one member with waiting=200 was entirely coordination wait; another with waiting=10600 was entirely time-window wait.

There is currently no way to separate them from the response. If you need that, we can add a `coordination_waiting_time` field - a small, well-scoped change.

Q3. What exactly happens with member pickup / delivery loads?

Fully independent, exactly like ordinary jobs. Each member's load consumes only its own vehicle's capacity. The group object has no load fields at all (validation rejects them there); loads may only be declared per member.

Verified with your exact example: member A delivery=10 on a capacity-15 vehicle, member B delivery=20 on a capacity-25 vehicle -> both assigned.

When a member's load exceeds the capacity of its only eligible vehicle, the entire group goes unassigned.

Q4. What happens when the solver finds several valid meeting windows?

Which one does it prefer?

None of the listed options - v1 has no dedicated meeting-time objective.

The window is anchored at:

max(intersection of member time-window starts,
earliest instant every member can be reached)

then given a randomized offset (capped at 4h) so successive iterations explore different meeting times. The constraint enforces only the UPPER bound.

Which iteration's plan ultimately wins is decided by the standard solution acceptance criteria - total travel cost, unassigned penalties, vehicle costs.

Waiting time is NOT part of the objective by default.

So the practical tendency is "earliest feasible", but it is neither strictly

earliest nor waiting-minimizing.

One rendering consequence worth knowing: routes containing coordination members SKIP the usual "delay departure to compress waiting" optimization (it would break the committed meeting time). Such routes therefore show larger waiting_time values than comparable ordinary routes.

Q5. Does group-level service apply after the synchronized arrival or before it?

After. Timeline under last_arrival (the default):

arrive -> wait (until sync instant) -> setup -> service -> depart

The sync instant = the maximum, across all members, of their time-window-adjusted earliest arrival.

Each member finishes at: syncInstant + own setup + own service

So with heterogeneous durations, members depart at different times (e.g. the inspector leaves before the installer). This is intended.

Q6. How is setup treated with last_arrival?

Does setup happen before or after synchronization?

After. Verified with setup=120, service=600:

veh=10 arrival=29400 waiting=0 -> $29400 + 120 + 600 = 30120$

veh=20 arrival=29200 waiting=200 -> $29400 + 120 + 600 = 30120$

Precisely: what gets aligned is the start of the combined setup+service action block. If members have DIFFERENT setup durations, service proper begins at different times, offset by that setup difference.

If you require service proper to start simultaneously, tell us - that is a semantic choice we can change.

Q7. Does group-level time_windows get intersected with every member's individual window?

No - it is REPLACEMENT, not intersection.

If a member specifies time_windows, it fully replaces the group default. Only when the member omits it does the group value apply.

Verified: group window [30000,31000] with member window [50000,51000] produced a result at 50000 - the group window was ignored entirely, with no error raised.

Worth highlighting in your test plan, because "the group window is a hard outer bound" is a natural but incorrect assumption.

Q8. If a member has multiple time windows, how does synchronization choose between them?

There is no explicit selection logic - it emerges, and it has a known blind spot.

Each member independently picks the earliest window feasible for its own arrival; the group's spread is then validated.

When a common window is the only option this converges correctly. Verified with member A on [[29300,31000],[50000,60000]] and member B on [[50000,60000]]:

A could not reach the early window, so both landed on 50000.

BLIND SPOT: if A CAN reach its early window and B cannot, A takes the early one, the spread check fails, and the whole group is rejected even though a later window works for both. Reproduced (2 unassigned).

This is the optimality trade-off the design doc records for the committed-meeting-window approach; removing it requires the coupled search planned for v2.

Q9. Can two members have exactly the same vehicle eligibility?

If yes, how does the optimizer select two distinct vehicles?

Yes, identical eligibility is allowed. Three mechanisms separate them:

1. A soft penalty (100000 per collision) steering insertion away from a route that already holds a member of the same group.
2. A hard post-check: if any route ends up with 2+ members of the same group, the whole group is rolled back.
3. A MaxCost x 5 dispersion penalty in the solution objective.

Verified: two members with NO skills at all (identical eligibility) consistently land on different vehicles.

Q10. What happens if only one vehicle satisfies both members' skills?

The whole group goes unassigned - two distinct vehicles cannot be obtained.

Q11. What happens with `arrival_sync_tolerance_seconds = 0`?

Accepted. It requires the members' time-window-adjusted earliest arrivals to be EXACTLY equal.

Without a shared window this almost always fails (a 200s difference was rejected).

WITH a shared window pinning both to the same instant it does work. Verified:

tolerance=0 plus group window [40000,60000] assigned successfully, both members at service start 40000.

Q12. Is there a maximum allowed tolerance?

This is a good point, an upper bound is actually needed because a huge value would result in the whole group unassigned. Tentatively let's set it as 86400 (one day) seconds

Q13. Is service allowed to be 0?

Yes. Allowed and verified working.

Q14. Is setup allowed to be 0?

Yes - it is the default value.

Q15. Are negative service/setup values rejected?

Yes, at the JSON layer, with a clear message:

cannot unmarshal number -5 into Go struct field

CoordinateJobMember.coordinate_jobs.jobs.service of type uint64

Q16. Are negative/empty skills rejected?

No - both are accepted, with no validation.

Negative values are legal identifiers: skill -3 correctly matched a vehicle carrying skill -3. Empty skills means no restriction.

Note these are literal ids, the same as zones - where -1 is a real zone id, not a wildcard.

Q17. Is jobs allowed to contain duplicate member descriptions?

Yes. description is not validated at all. Only id must be unique.

Q18. If one member's vehicle becomes unavailable, does the entire group get re-synchronized?

Yes. Verified: after removing vehicle 10, member 5011 moved to vehicle 30 and both members re-aligned at service start 29200.

Mechanism: losing a vehicle strands a member, which causes the entire group to be pulled off its routes and re-inserted as a unit under a freshly chosen meeting window.

Q19. If the previous meeting time becomes impossible, does the solver freely choose a new meeting time?

Yes, freely. Meeting windows live only in solver state - never written to the model, never persisted across requests. They are cleared when a group is disturbed and re-chosen on re-insertion.

For transparency: we did NOT implement seeding the window from the previous plan's arrival times (an optional item in the original plan). So an undisturbed group keeps its original vehicles via initial-route seeding, but any group that gets re-planned picks an entirely new meeting time.

Q20. Can shipment_changes or other existing reoptimization operations reference a coordinate member ID?

- job_changes.add / job_changes.modify referencing a member or group id
-> explicitly rejected.
- job_changes.remove -> whole-group semantics: the group id, or all member ids, removes the group; a partial subset is an error.

- shipment_changes -> not guarded, and intentionally so. Coordinate members are materialized internally as ordinary single jobs, so they occupy the same id namespace as ordinary jobs. Ordinary job ids and shipment pickup/delivery ids have always been separate namespaces on this platform and are permitted to coincide. Holding coordinate members to a stricter rule than ordinary jobs would be inconsistent, so member ids are unique within the job id namespace - the same guarantee any ordinary job gets. (The tech doc's validation table currently overstates this as "globally unique across all jobs, shipment steps, groups and members". That wording will be corrected; the behavior is correct as implemented.)

30. My recommended E2E test

If I were testing this feature from scratch, **this is the first complete test I would build.**

Input

None

Group 501

Location = Customer

2 technicians

Technician 1:

skill = electrician

Technician 2:

skill = HVAC

Vehicles:

None

V1 → electrician

V2 → HVAC

Time window:

None

09:00-17:00

Tolerance:

None

300 sec

Expected

None

V1

↓

Technician 1

↓

Customer

arrival = ~10:00

V2

↓

Technician 2

↓

Customer

arrival = ~10:00

Then automatically assert:

None

✓ 2 members assigned

✓ different vehicles

✓ same location

✓ both have coordination_group_id = 501

✓ arrival difference <= 300 seconds

✓ both satisfy their skills

- ✓ both satisfy time windows
- ✓ no member is unassigned
- ✓ service_start_rule behaves correctly
- ✓ response contains no unexpected group step

Then deliberately break one thing at a time:

```
None
Only 1 eligible vehicle
    ↓
WHOLE GROUP unassigned

Arrival difference > tolerance
    ↓
WHOLE GROUP unassigned

Non-overlapping time windows
    ↓
WHOLE GROUP unassigned

Duplicate IDs
    ↓
400

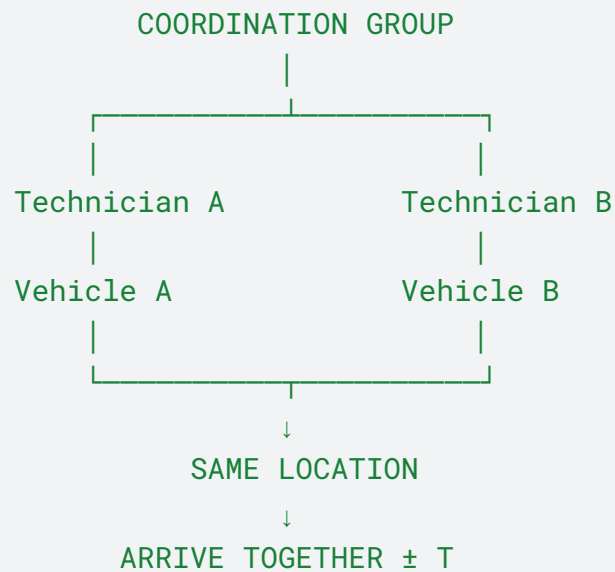
Invalid tolerance
    ↓
400
```

That gives you a very strong foundation.

The mental model I want you to keep

Think of `coordinate_jobs` as:

None



The **group** controls the coordination.

The **members** are real jobs.

Each member gets its **own vehicle, skills, service, time window and load**.

The group guarantees:

None

`N technicians`
`+`
`N distinct vehicles`
`+`
`same location`
`+`
`arrival spread <= tolerance`
`+`
`ALL or NOTHING`

That is the core of the entire feature.

And importantly, the design has already resolved three things that might otherwise have been Engineering questions: **service timing** (`last_arrival` default / `first_arrival` option), **late technicians** (re-optimize rather than live holding), and **costing** (each technician is costed normally).